CS1302 GUEST LECTURE

What's Inside an AI Agent?

```
agent =
    LLM
+ tools
+ harness
```

LLMs, tools and harnesses.
Then we build one in Python.

Bexultan Karimtayev • Fri 6 Nov 2026 • LI-4109

ABOUT ME

A year ago, I was in your seat

2025

Learned Python in CS1302, just like you.

2026

PALSI leader for CS1302, CS1302A and CS2310.

Summer 2026

Built two Python packages with an AI agent: traceback-coach and hermes-acp-sdk, both on PyPI.

Hermes writes code, runs it and fixes its own bugs.

So how does it actually work?

LIVE DEMO

First, let's watch an agent work

```
Task: Write is_prime in primes.py and test it
```

While it runs, watch for three things:

`[step 1] …`

The LLM asking to use a tool.

`allow? [y/n]`

Me deciding if its code may run.

`[result] …`

What the tool gave back.

THE DEMO, FROZEN

What you just saw

```
Task: Write is_prime in primes.py and test it
[step 1] write_file(path, content)  ← the LLM asks for a tool
[result] Wrote 107 characters to primes.py
[step 2] run_python(code)
--- allow? [y/n] y  ← a human says yes
[result] [0, 1, 2, 3, 5, 7, 11]  ← wrong: 0 and 1 are not prime
[step 3] write_file(path, content)  ← it fixes its own code
[result] Wrote 142 characters to primes.py
[step 4] run_python(code)
--- allow? [y/n] y
[result] [2, 3, 5, 7, 11]  ← correct
Agent: Done. My first version said 0 and 1 are prime, which is wrong…
```

Who wrote the code? Who ran it? How did it notice the bug?

BEFORE THE THEORY

Why look inside the box?

Use Hermes better

Know what it can see and what it can’t, so you ask it the right way.

Catch its mistakes

It guesses, checks and can be wrong. You are part of the loop.

Build your own

It is about 43 lines of Python you already know.

TODAY

Three words, then four steps

1 • Three words

LLM, tool and harness: what they mean and how they fit together.

2 • Four steps

We build the agent you just saw, from 5 lines to 43.

3 • Your turn

Run it tonight, then teach your agent something new.

WORD 1 • LLM

An LLM turns text into more text

YOU SEND

“What files are in my folder?”

→

LLM

Large Language Model: predicts the next words

→

IT REPLIES

“Sorry, I can’t see the files on your computer.”

On its own, an LLM has no hands:

- Can’t run code
- Can’t open your files
- Can’t check the web

WORD 2 · TOOL

A tool is a function the LLM can ask us to run

If the LLM is the brain, tools are its hands.

</>

`run_python`

Run some Python and see what it prints.

`read_file`

Read a file from your computer.

`search_web`

Look something up online.

WORD 2 · TOOL

How the LLM uses a tool

1 We send a menu

```
name:  run_python
does:  runs Python code
needs: code
```

2 It replies with a request

```
{"name": "run_python",
 "arguments": {
  "code": "print(2**200)"}}
```

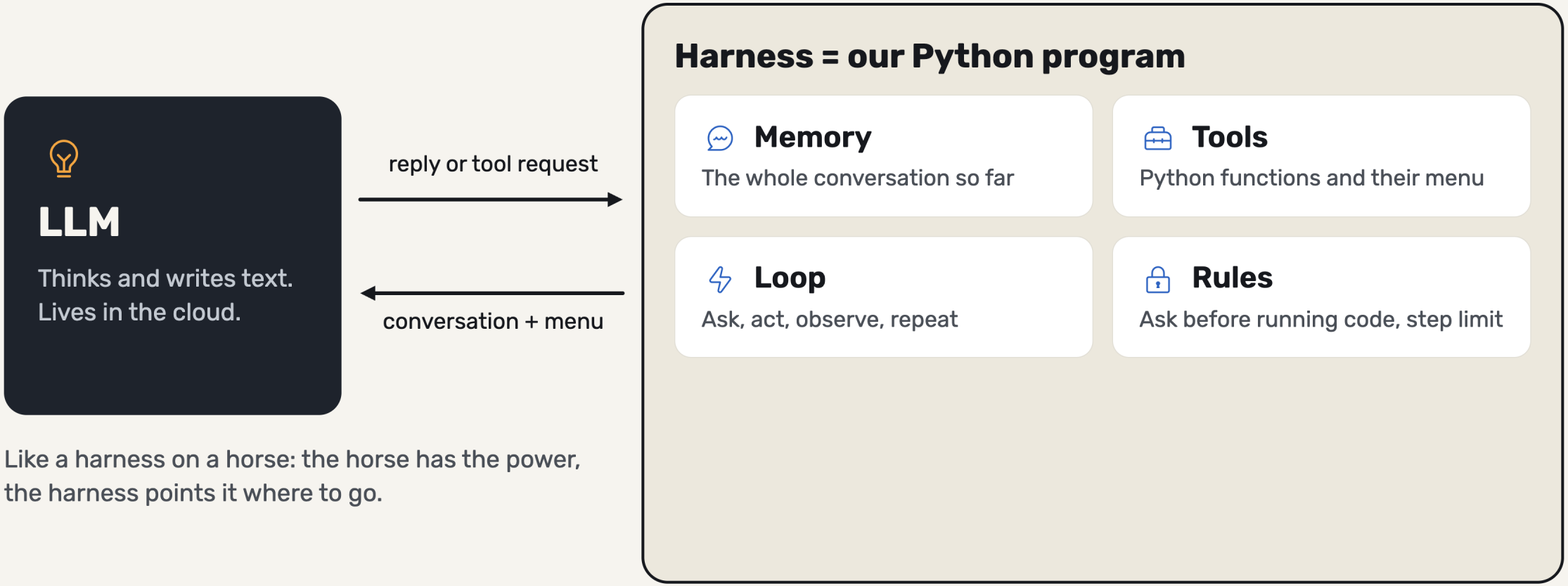
3 Our code runs it

```
run_python(
  "print(2**200)")
→ 1606938044258990275…
```

The LLM only asks. Our code does the work.

WORD 3 • HARNESS

A harness is the code around the LLM



Like a harness on a horse: the horse has the power, the harness points it where to go.

Agent =
LLM + tools + harness

Hermes, Claude Code and Cursor all work this way.
Today we build our own.

LET'S BUILD IT

Four steps, four files you can run

STEP 0

Just the LLM

Text in, text out. No hands yet.

`step0_chat.py`

5 lines of code

STEP 1

Add a tool

The LLM can ask to run code.

`step1_tool.py`

23 lines of code

STEP 2

Close the loop

Results go back. Now it's an agent!

`step2_loop.py`

25 lines of code

STEP 3

Grow the harness

More tools, rules and limits.

`mini_agent.py`

43 lines of code

STEP 0 · STEP0_CHAT.PY

Just the LLM: text in, text out

```python
from llm import client, MODEL  # llm.py connects to the LLM; the key stays hidden in .env

question = input("You: ")
response = client.chat.completions.create(model=MODEL,
                                          messages=[{"role": "user", "content": question}])
print("AI:", response.choices[0].message.content)
```

```
You: What files are in my folder?
AI: Sorry, I can't see the files on your computer.
```

A chatbot in a few lines. It can talk, but it can't do anything.

STEP 1 • STEP1_TOOL.PY

Add a tool: a function plus a menu

```
def run_python(code):  # the tool: an ordinary Python function
    if input(f"\n--- run this code? ---\n{code}\n[y/n] ") ≠ "y":
        return "The user said no."
    r = subprocess.run([sys.executable, "-c", code], capture_output=True, text=True, timeout=30)
    return r.stdout + r.stderr or "(no output)"

TOOL_MENU = [{"type": "function", "function": {  # the menu: name, what it does, what it needs
    "name": "run_python",
    "description": "Run Python code and return what it prints",
    "parameters": {"type": "object",
                   "properties": {"code": {"type": "string"}},
                   "required": ["code"]}}}]
```

STEP 1 • STEP1_TOOL.PY

It asks, but never sees the result

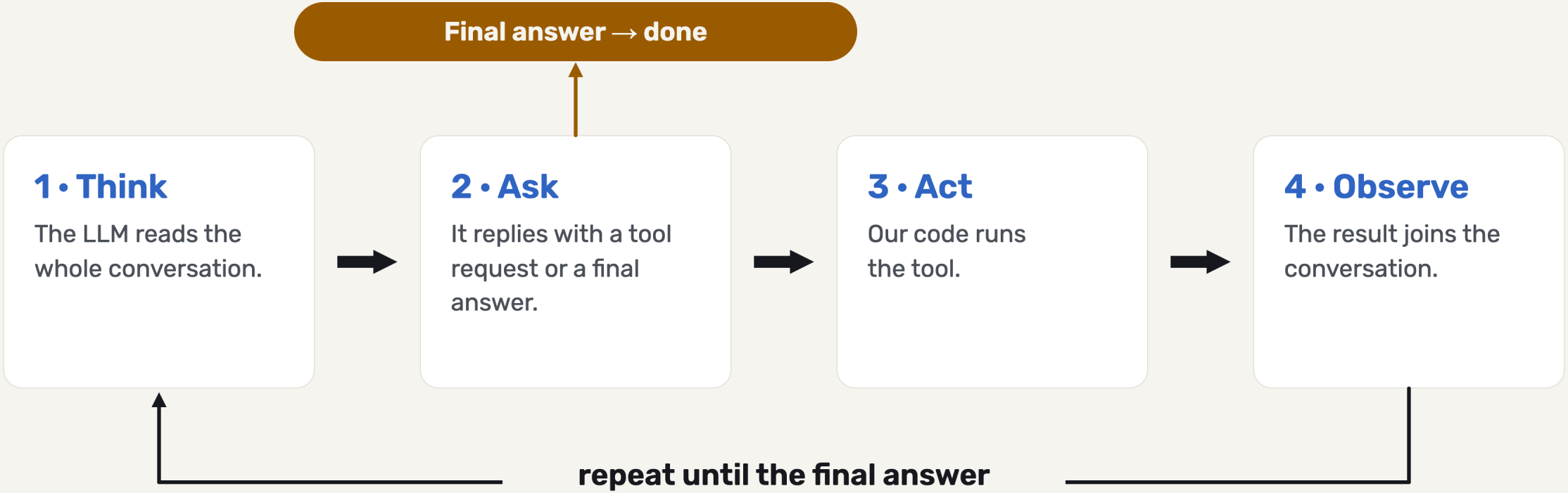
```
question = input("You: ")
response = client.chat.completions.create(model=MODEL, tools=TOOL_MENU,
                                          messages=[{"role": "user", "content": question}])
reply = response.choices[0].message
if reply.tool_calls:  # the LLM answered with a tool request instead of text
    call = reply.tool_calls[0]
    print("The LLM asks for:", call.function.name, call.function.arguments)
    print("Our code runs it and gets:", run_python(**json.loads(call.function.arguments)))
else:
    print("AI:", reply.content)
```

```
The LLM asks for: run_python {"code": "print(2 ** 200)"}
Our code runs it and gets: 1606938044258990275541962092341162602522202993782792835301376
```

Problem: the result never goes back to the LLM.

STEP 2 • CLOSE THE LOOP

Send the result back, and repeat



STEP 2 • STEP2_LOOP.PY

The loop in Python: now it's an agent

```
messages = [{"role": "user", "content": input("You: ")}]  # the conversation so far
while True:
    response = client.chat.completions.create(model=MODEL, messages=messages, tools=TOOL_MENU)
    reply = response.choices[0].message
    messages.append(reply.model_dump(exclude_none=True))  # remember what the LLM said
    if not reply.tool_calls:  # no tool request → this is the final answer
        print("AI:", reply.content)
        break
    for call in reply.tool_calls:  # run every tool it asked for...
        result = run_python(**json.loads(call.function.arguments))
        print("[result]", result)
        messages.append({"role": "tool", "tool_call_id": call.id, "content": result})  # ...show it
```

New problem: what if it never stops, or a tool crashes?

STEP 3 • MINI_AGENT.PY

Grow the harness

`write_file read_file`

Two more tools: now it can save and read files.

`max_steps=10`

A step limit, so a stuck LLM can’t loop forever.

`try / except`

Errors go back to the LLM, so it can fix them.

`timeout=30`

Code that never ends is stopped after 30 seconds.

`allow? [y/n]`

A human still approves every piece of code.

`system prompt`

Standing orders: test your code, then summarise.

43 lines of code in total. That’s the agent from the demo.

EXAMPLE RUN • OFFLINE_DEMO.PY

One task, step by step

Step	The LLM asks for	Our code sends back
1	write_file: save primes.py	Wrote 107 characters to primes.py
2	run_python: test is_prime	[0, 1, 2, 3, 5, 7, 11] (wrong: 0 and 1 are not prime)
3	write_file: save a fixed primes.py	Wrote 142 characters to primes.py
4	run_python: test again	[2, 3, 5, 7, 11] (correct)
5	No tool: final answer	“Done. My first version said 0 and 1 are prime…”

Nobody told it about the bug. It read the test output in the conversation and fixed it.

FROM 43 LINES TO A REAL AGENT

Real harnesses are bigger, but the loop is the same

	Our harness	Real coding agents
Tools	3	Dozens: edit, search, terminal, web
Memory	One task	Remembers across sessions
Safety	A y/n question	Sandboxes and permission rules
Long tasks	Up to 10 steps	Plans, summarises and keeps going

YOUR TURN

Try it tonight

1
```
pip install openai
```

2
```
open -e .env   # paste key
```

3
```
python check_llm.py
```

Then step0, step1, step2 and mini_agent.py. It uses CityU's DeepSeek server by default; any OpenAI-compatible API works (see the README). No key? Run offline_demo.py.

Code: [link to the code]

Then make it yours

- Add a list_files tool
- Add an "always allow" answer
- Print how many steps it used
- Ask it to build a small game

Questions?

An agent is an LLM, some tools and a harness.
You already know enough Python to build one.

Bexultan Karimtayev • Code: [link to the code]